

LA CIGOGNE D'AILLEURS

AI Interior Visualization & Remove-Only Editing Platform

CLIENT TECHNICAL DOSSIER

Prototype status: Phase 5.2.28

Purpose of this document

Technical explanation of the current prototype, its architecture, AI pipeline, object-removal strategy, validation system, current limitations, and the recommended roadmap for demonstrating the working prototype on a smartphone.

Product principle: the Remove AI feature is designed as a true object-removal workflow. The selected furniture is not intentionally replaced by another piece of furniture. The system reconstructs the room/background that should exist behind the selected object.

Prepared for a technically knowledgeable client

Current implementation documented from the Phase 5.2.28 project source and changelogs.

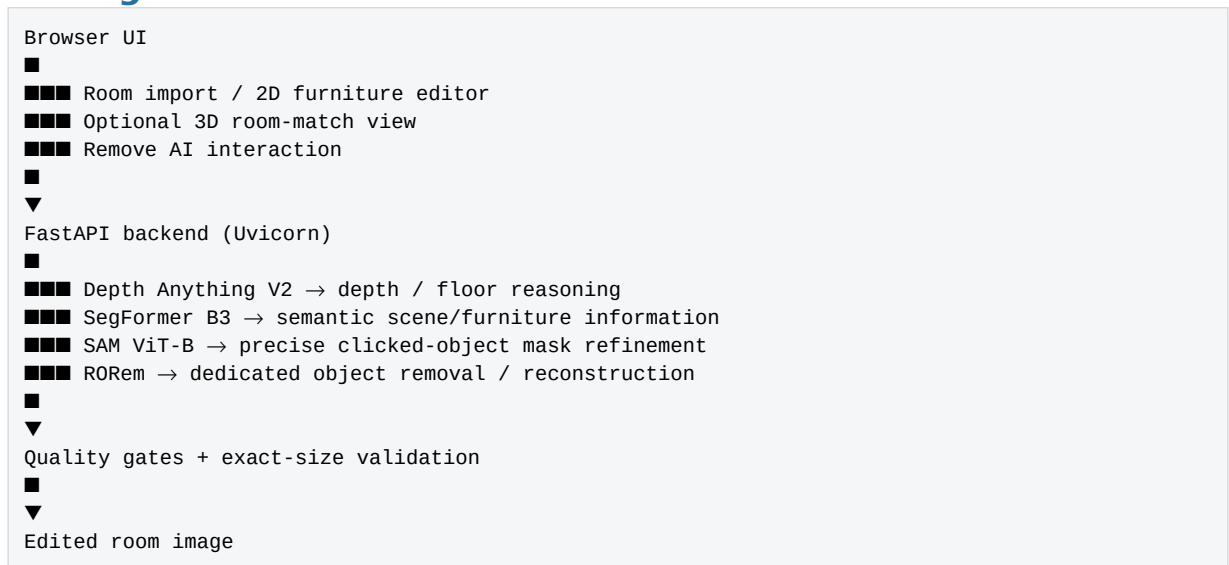
1. Executive summary

La Cigogne D'Ailleurs is a browser-based interior visualization prototype that combines 2D room editing, AI scene analysis, furniture placement, an optional 3D room-matching view, and AI-powered object removal.

The most technically important component is Remove AI. A user imports a room photo, activates AI removal, clicks a real furniture object, confirms the selection, and the backend reconstructs the hidden room context. The current large-object engine is RORem, operated through Diffusers and a 512×512 object-centred working crop.

Phase 5.2.28 is primarily a reliability release. It keeps the strict visual-quality gate but introduces a narrow recovery band for candidates that are semantically clean and genuinely remove the target while missing the normal seam threshold by a small amount. It also protects the frontend history/working image when an AI operation fails or returns incompatible dimensions.

Current high-level architecture:



2. Product scope and current capabilities

The current interface exposes the following main functions:

- Import a room image in JPEG, PNG, WebP and common image formats.
- Analyze the room with AI and expose optional floor, depth and zone diagnostics.
- Add furniture from the built-in catalogue and manipulate it in the 2D room editor.
- Use an optional 3D room-match mode based on Three.js, with GLB/GLTF model loading.
- Select a real furniture object using the AI removal workflow.
- Preview the detected object mask before confirming removal.
- Undo successful AI removals.
- Export the edited scene as PNG.

The project therefore already contains more than a single image-inpainting demo: it is a prototype platform with an AI scene-understanding layer, an interaction layer, a 2D editing layer and a 3D visualization layer.

3. Technical architecture

Frontend

- HTML/CSS/JavaScript browser application.
- js/app.js: main room editor, image state, object selection, 2D interaction, export and UI state.
- js/ai.js: AI API communication, room analysis and health checking.
- js/eraser.js: Remove AI selection, mask display, confirmation, image upload, result loading and undo history.
- js/phase3.js: 3D room-match/visualization layer.
- js/phase3e.js: additional phase-3 enhancements.
- js/furniture-data.js: furniture catalogue data.

Backend

- FastAPI application served by Uvicorn.
- Python / PyTorch ecosystem for the AI pipeline.
- Transformers for Depth Anything V2 and SegFormer.
- SAM for point-guided object-mask refinement.
- Diffusers for RORem inference.
- OpenCV + Pillow + NumPy for image/mask processing and supporting reconstruction/validation operations.
- CORS is enabled for browser-to-API communication.

Important deployment characteristic

The current prototype is GPU-backed. RORem is intended to run on CUDA for the client-demo quality path. The ZIP contains project code and configuration, not multi-gigabyte model weights; the required Hugging Face weights are downloaded/cached by the runtime.

4. Remove AI pipeline — detail by detail

Step 1 — User imports the room

The browser keeps the room image as the current working image. The Remove AI workflow uses the image at its native working dimensions rather than intentionally replacing the room with a low-resolution square image.

Step 2 — User clicks an object

The frontend sends the image plus the clicked coordinates to /select-mask. The backend uses SegFormer to identify a likely furniture class and then refines the physical object silhouette using SAM/point-guided mask refinement. The semantic model supplies class/context; SAM supplies the actual removal silhouette.

Step 3 — User confirms the removal

The frontend sends the current room JPEG plus the selected grayscale mask to /inpaint. The backend validates the image and mask dimensions and chooses the large-object Remove AI path when

appropriate.

Step 4 — Object-centred RORem crop

The selected object is converted into an object-centred square crop with contextual pixels around it. This is deliberate: the base RORem checkpoint is designed around its native 512-pixel regime, so the selected object occupies much more useful model resolution than it would in a full-room 1408×768-to-512 adaptation.

Step 5 — Internal mask halo

The confirmed user mask is kept as the product boundary. Internally, RORem receives a slightly dilated mask to help eliminate object edges and contact shadows. The dilation is adaptive: larger objects receive a wider internal halo while smaller objects keep a tighter one.

Step 6 — RORem generation

The current Remove AI prompt explicitly describes photorealistic reconstruction of the existing room and asks for complete removal with no replacement object. The negative prompt also penalizes furniture and replacement objects.

Step 7 — Candidate scoring

A generated candidate is not automatically accepted. The system evaluates the candidate using multiple signals: boundary/seam quality, surviving furniture/hallucination penalty, target-change score and a lightweight structural residual score. The goal is to distinguish genuine background reconstruction from a candidate that merely changes the pixels.

Step 8 — Refinement

The best candidate can receive additional RORem removal passes. A refinement is accepted only if it materially improves semantic cleanliness, structural residual or overall quality. This protects a good result from unnecessary drift.

Step 9 — Quality gate

Phase 5.2.28 uses a normal strict gate plus a narrow recovery band. A candidate that fails both is rejected rather than shown as a fake successful removal.

Step 10 — Exact-size result and history

The frontend checks that the returned image dimensions match the current working room. Only after a successful dimension check and image load is the edit committed to history. A failed result leaves the working room untouched.

5. Phase 5.2.28 quality gates

Gate	Condition	Purpose
Normal target-change	≥ 0.28	Confirms the selected region actually changed.
Normal furniture penalty	≤ 0.40	Rejects obvious furniture/ghost-object regeneration.
Normal seam	≤ 50	Controls boundary discontinuity.
Recovery target-change	≥ 0.32	Requires stronger evidence of genuine removal.
Recovery furniture penalty	≤ 0.20	Recovery is only for very clean candidates.

Gate	Condition	Purpose
Recovery residual structure	≤ 0.45	Limits furniture-like structural residue.
Recovery seam	≤ 58	Allows a small boundary miss when the other signals are strong.

Why not simply remove the gate? Because a 503 in this prototype means the system refused to present a reconstruction it judged insufficient. That is preferable to silently presenting a hallucinated sofa/bed or a visibly smeared region to a client.

6. Development history — how the Remove AI evolved

Phase	Main lesson / change
5.2.15	Surface-guided reconstruction was prohibited from winning the final large-object result after producing stretched/blurred results.
5.2.16	Product requirement clarified and enforced: REMOVE-ONLY; no furniture replacement.
5.2.17	Attempted surface-aware LaMa routing for wall/rug/floor; visual result still showed large-object artifacts.
5.2.18	Moved toward true reconstruction with deterministic wall handling and localized rug/floor reconstruction.
5.2.19	Fixed a backend PIL/NumPy crash causing browser 'Failed to fetch'.
5.2.20	Introduced RORem as a dedicated object-removal engine.
5.2.21	SmartEraser integration was attempted, but its custom pipeline was unavailable in the installed generic Diffusers environment.
5.2.22	Corrected RORem inference toward the official AutoPipeline/512 regime.
5.2.23	Fixed RORem loader configuration so inference could actually run.
5.2.24	Fixed missing target-change scoring and restored candidate quality evaluation.
5.2.25	Moved to an object-centred 512×512 crop and tightened candidate acceptance.
5.2.27	Added adaptive dilation, more candidate search, structural residual scoring and conservative refinement passes.
5.2.28	Reliability pass: narrow high-confidence recovery band + safer frontend history/dimension handling.

7. Backend API surface relevant to the prototype

Endpoint	Role
GET /health	Checks backend version, device/GPU information and model readiness.
POST /analyze	Runs room analysis: depth, semantic masks, floor information and scene metadata.
POST /select-mask	Takes an image + click coordinates and returns the selected object's label and mask.
POST /inpaint	Main Remove AI endpoint: image + mask → validated remove-only reconstruction.
POST /remove	Backward-compatible one-click removal endpoint using precise selection.
GET /inpaint-status	Reports current large-object removal strategy and quality thresholds.

8. Showing the working prototype on a phone

Important: for a client demonstration, you do not need to deploy the AI model to a public cloud first. The simplest and safest demonstration is to keep the GPU inference on your PC and let the phone act as the browser client.

Option A — Same Wi-Fi / local demonstration (recommended for the first client meeting)

Connect the phone and the PC to the same Wi-Fi network. The phone opens the frontend from the PC, while the FastAPI backend continues to run on the PC's RTX GPU.

```
Terminal 1 – backend
cd "C:\Users\pc\Desktop\La Cigogne D'Ailleurs creative\server"
uvicorn main:app --reload --host 0.0.0.0 --port 8000

Terminal 2 – frontend
cd "C:\Users\pc\Desktop\La Cigogne D'Ailleurs creative"
python -m http.server 5500 --bind 0.0.0.0
```

Then on Windows run `ipconfig` and find the PC's IPv4 address, for example 192.168.1.25. On the phone open `http://192.168.1.25:5500`.

The current frontend derives the API host from the browser hostname and uses port 8000, so the local-network architecture is already close to this mode. If Windows Firewall blocks access, allow inbound TCP traffic for ports 5500 and 8000 on the private network.

Option B — Remote client demonstration through a tunnel

If the client is not on the same network, keep the GPU server on your PC and expose the local web application through a secure tunnel. Cloudflare Tunnel is a practical option: Cloudflare documents an outbound-only connection from cloudflared to Cloudflare, so the origin does not need an exposed public IP or inbound port.

For a polished remote demo, the recommended architecture is one public HTTPS application endpoint with the browser frontend and API routed through the same origin (for example, `/` for the UI and `/api/*` for FastAPI). This avoids browser mixed-content issues and makes the phone configuration simple.

The current prototype has the API on a separate port. Therefore, before the remote demo, the next engineering step should be a small 'Demo Mode' release that makes the API base URL configurable or places the frontend and API behind one reverse-proxy origin. Do not publicly expose the current GPU backend without an access-control layer.

Option C — Real production deployment

A production SaaS deployment is a different phase. The frontend can be hosted on a static/web platform, but the Remove AI backend requires a CUDA GPU with enough VRAM and cached model weights. A cloud GPU backend or dedicated GPU server should therefore be planned only after the client validates the product concept.

9. Recommended client demonstration script

- 1 Start with the result: open the room photo and say that the system can edit the photographed room without rebuilding the entire scene.
- 2 Analyze: show the AI analysis/depth/zone capabilities briefly, but do not spend the first minutes on technical details.
- 3 Remove a large object: click the bed/sofa, show the detected mask, confirm, and wait for the reconstruction.
- 4 Show that it is truly remove-only: point out that surrounding lamps, shelves, doors and other furniture remain outside the selected mask.

- 5 Do a second removal: remove a table/nightstand from the already-edited image. This demonstrates sequential editing.
- 6 Do a third small-object removal: demonstrate that the workflow is not limited to one single edit.
- 7 Then explain the technical architecture: SegFormer + SAM for selection, RORem for reconstruction, quality gates for acceptance, and exact-size validation for reliability.
- 8 Finish with the roadmap: production hosting, configurable API/demo mode, authentication, faster inference and further reconstruction-quality improvements.

Do not present the current prototype as production SaaS. Present it as a technically functional prototype whose core interaction and AI removal pipeline are already demonstrated.

10. Recommended next engineering roadmap

Priority	Work	Why now
P0	Demo Mode / phone-ready configuration	Make local-LAN and remote-demo deployment predictable; configurable API URL;
P0	Reliability test matrix	Run 20–50 removals across large/small furniture and sequential edits; measure 50
P1	Quality improvement	Tune crop context, mask dilation, candidate ranking and residual detection using
P1	Demo UX	Mobile responsive layout, clearer progress state, 'AI removing...' stage, result/und
P1	Security for remote demo	Access control, request limits, upload limits and non-public backend configuration
P2	Performance	Model warm-up, VRAM management, candidate-count tuning and optional lower-c
P2	Production architecture	Dedicated GPU inference service, persistent storage, job queue, logging and moni
P3	Commercial productization	Accounts, projects, billing, furniture catalogue, collaboration and scalable cloud in

11. Current limitations and honest technical assessment

- A single photograph does not contain the exact pixels hidden behind furniture. Any removal model must reconstruct plausible content from visible context; perfect pixel recovery is impossible without additional views.
- Large masks are inherently harder than small masks, especially when the removed object covers a large portion of the wall/floor/rug boundary.
- The current base RORem workflow operates at a 512×512 working crop. Higher-resolution/mixed-resolution model strategies can be explored later, but should be benchmarked before integration.
- Quality metrics are proxies. A low numerical penalty does not guarantee a human will consider every reconstruction perfect; visual benchmark testing remains necessary.
- Inference is computationally expensive because multiple RORem candidates/refinement passes may be evaluated.
- The ZIP does not contain multi-gigabyte model weights. A connected machine must be able to download/cache the required models before the first use.
- The current prototype is not yet a public production service: authentication, abuse protection, persistent user storage, billing and scalable GPU orchestration are outside Phase 5.2.28.

12. Phone-demo checklist

- PC and phone connected to the same Wi-Fi for the simplest demo.
- RTX GPU driver/CUDA environment tested.
- RORem model already downloaded and warmed up before the client arrives.
- Backend started with `--host 0.0.0.0` for LAN access.
- Frontend started with `--bind 0.0.0.0`.
- Windows Firewall permits ports 5500 and 8000 on the private network.
- Phone opens the PC IPv4 address on port 5500.
- Prepare one known-good room image and 3–5 objects to remove.
- Have the PC charger/power connected; keep the machine awake during the demo.
- If using a remote tunnel, test the complete phone flow from a different network before the client meeting.

Recommended demo strategy: preload/warm the models, use a known-good 1408×768 or higher-quality room image, and demonstrate three sequential removals. This shows the strongest part of the current prototype without pretending that every arbitrary photo will always pass the quality gate.

13. Project source basis and external technical references

This dossier is based primarily on the Phase 5.2.28 project source files and changelogs included in the project ZIP. The exact implementation terms used here follow the project's own naming: Remove AI, RORem, SAM, SegFormer, Depth Anything V2, strict gate, recovery band and exact-size validation.

For RORem behavior, the project history refers to the official RORem inference regime. For Cloudflare Tunnel, current Cloudflare documentation (September 2026) describes cloudflared as an outbound-only tunnel and documents published hostnames mapping to local services. The official documentation also distinguishes quick tunnels for development/testing from managed tunnels for production.

Useful official references:

Cloudflare Tunnel setup: <https://developers.cloudflare.com/tunnel/setup/>

Cloudflare Tunnel overview: <https://developers.cloudflare.com/tunnel/>

Cloudflare Tunnel downloads: <https://developers.cloudflare.com/tunnel/downloads/>

Project source files: README.md, START_HERE.txt, server/main.py, js/ai.js, js/eraser.js, js/app.js, js/phase3.js, js/furniture-data.js.

14. Final technical position of Phase 5.2.28

Phase 5.2.28 is a functional prototype milestone, not the final production architecture. The core technical achievement is that the Remove AI feature now performs genuine sequential object removal using a dedicated object-removal model, while preserving the original room outside the confirmed mask and protecting the working image/history when a result fails validation.

The most rational next step is therefore Demo/Deployment readiness: make the application phone-ready, make the API endpoint configurable, test the removal pipeline systematically, and then continue quality tuning using a fixed benchmark. Once the client validates the concept, the project can move toward a real hosted GPU service.